

Chương 1

Từ Monolithic đến Microservice và DDD

1.1 Kiến trúc đơn khối (Monolithic Architecture)

1.1.1 Khái niệm và Bản chất Kỹ thuật

Kiến trúc Monolithic không chỉ đơn thuần là việc đóng gói mã nguồn vào một tệp tin thực thi (như .war hay .jar), mà còn đại diện cho một tư duy thiết kế tập trung. Trong mô hình này, toàn bộ logic nghiệp vụ, giao diện người dùng, và các thành phần truy xuất dữ liệu được liên kết chặt chẽ (tightly coupled) trong một đơn vị duy nhất.

Sự gắn kết này có nghĩa là các thành phần chia sẻ chung bộ nhớ, tài nguyên CPU và thường là một cơ sở dữ liệu quan hệ duy nhất. Bất kỳ thay đổi nhỏ nào trong một module cũng đòi hỏi việc biên dịch lại và triển khai lại toàn bộ hệ thống.

1.1.2 Cấu trúc Phân lớp điển hình

Đa số các hệ thống Monolithic hiện đại được tổ chức theo mô hình ****Layered Architecture**** (Kiến trúc phân lớp):

- **Presentation Layer:** Xử lý giao diện người dùng và các yêu cầu HTTP.
- **Business Logic Layer:** Chứa các quy tắc nghiệp vụ cốt lõi của hệ thống.
- **Data Access Layer (DAL):** Chịu trách nhiệm giao tiếp với cơ sở dữ liệu.

Mặc dù được phân lớp về mặt logic, nhưng ở cấp độ vật lý, chúng vẫn chạy chung một tiến trình (process).

1.1.3 Ưu điểm và Hạn chế trong Vận hành

Ưu điểm:

- **Đơn giản trong quản lý:** Quá trình kiểm thử tích hợp (End-to-End testing) diễn ra thuận lợi do không có độ trễ mạng giữa các thành phần.
- **Hiệu suất tối ưu:** Các lời gọi hàm trực tiếp trong bộ nhớ luôn nhanh hơn việc gọi qua API hoặc Message Broker.
- **Nhất quán dữ liệu (ACID):** Việc sử dụng một DB duy nhất giúp đảm bảo tính toàn vẹn dữ liệu thông qua các Transaction truyền thống.

Hạn chế (The Big Ball of Mud):

- **Rào cản về quy mô (Scalability):** Không thể mở rộng riêng biệt một module bị nghẽn (ví dụ: module xử lý thanh toán), buộc phải nhân bản toàn bộ ứng dụng, gây lãng phí tài nguyên.
- **Sự trì trệ về công nghệ (Technology Lock-in):** Việc nâng cấp một thư viện hoặc ngôn ngữ mới là cực kỳ rủi ro vì nó ảnh hưởng đến tất cả các phần còn lại.

1.2 Kiến trúc Dịch vụ nhỏ (Microservices Architecture)

1.2.1 Triết lý thiết kế và Đặc điểm cốt lõi

Khác với Monolithic, Microservices tuân thủ triết lý "Làm một việc và làm thật tốt" (Do one thing and do it well). Mỗi dịch vụ là một thực thể độc lập, có vòng đời phát triển và triển khai riêng biệt.

Đặc điểm quan trọng nhất là **Bao đóng dữ liệu (Data Encapsulation)**: Mỗi microservice sở hữu cơ sở dữ liệu riêng (Database per Service), ngăn chặn việc truy cập dữ liệu trực tiếp từ các dịch vụ khác, từ đó giảm thiểu sự phụ thuộc.

1.2.2 Hệ sinh thái và Các thành phần hỗ trợ

Để một hệ thống Microservices vận hành ổn định, cần có các thành phần hạ tầng phức tạp:

- **API Gateway:** Điểm tiếp nhận duy nhất cho client, chịu trách nhiệm định tuyến, xác thực và giới hạn lưu lượng.
- **Service Discovery:** Cho phép các dịch vụ tìm thấy nhau trong môi trường mạng động (ví dụ: Netflix Eureka, Consul).
- **Circuit Breaker:** Cơ chế ngăn chặn lỗi dây chuyền khi một dịch vụ gặp sự cố (ví dụ: Hystrix hoặc Resilience4j).

1.2.3 Thách thức về Quản trị và Nhất quán dữ liệu

Mặc dù mang lại sự linh hoạt, Microservices đối mặt với bài toán **Nhất quán sau cùng (Eventual Consistency)**. Do dữ liệu bị phân tán, việc thực hiện các giao dịch phân tán (Distributed Transactions) trở nên cực kỳ khó khăn, thường phải áp dụng các mô hình như **Saga Pattern** để điều phối.

1.3 Phân tích so sánh: Khi nào nên chuyển đổi?

Việc lựa chọn kiến trúc không dựa trên xu hướng, mà dựa trên "Complexity-Value Curve" (Đường cong giá trị và độ phức tạp).

- Với các dự án khởi nghiệp cần MVP nhanh, Monolithic là sự lựa chọn tối ưu.
- Khi số lượng lập trình viên tăng lên (trên 20-30 người) và các yêu cầu nghiệp vụ trở nên quá chông chéo, việc phân rã sang Microservices trở thành yếu tố sống còn để duy trì tốc độ phát triển.

1.4 Thiết kế hướng tên miền (Domain-Driven Design - DDD)

1.4.1 Tại sao DDD là chìa khóa cho Microservices?

Một trong những lỗi lớn nhất khi triển khai Microservices là phân rã dịch vụ theo kỹ thuật (ví dụ: Service DB, Service UI). DDD giải quyết vấn đề này bằng cách tập trung vào nghiệp vụ cốt lõi.

1.4.2 Chiến lược thiết kế (Strategic Design)

Khái niệm quan trọng nhất là **Bounded Context** (Ngữ cảnh bao đóng). Nó xác định ranh giới logic mà trong đó một mô hình dữ liệu có ý nghĩa nhất định. Ví dụ: Từ "Sản phẩm" trong ngữ cảnh "Bán hàng" sẽ khác với "Sản phẩm" trong ngữ cảnh "Kho bãi". Việc xác định đúng các Bounded Context chính là bước đầu tiên để xác định ranh giới của các Microservices.

1.4.3 Context Mapping: Bản đồ tương tác

Context Map giúp mô tả mối quan hệ giữa các ngữ cảnh:

- **Shared Kernel:** Hai ngữ cảnh chia sẻ chung một phần mô hình.
- **Anticorruption Layer (ACL):** Lớp đệm giúp dịch vụ mới không bị ảnh hưởng bởi thiết kế lỗi thời của hệ thống cũ.

1.4.4 Thiết kế kỹ thuật (Tactical Design)

Để thực thi logic bên trong một Microservice, DDD cung cấp các công cụ:

- **Aggregate:** Một cụm các đối tượng dữ liệu được coi là một đơn vị để thay đổi dữ liệu.
- **Value Objects:** Các đối tượng không có định danh, chỉ chứa giá trị (như Địa chỉ, Tiền tệ).
- **Repositories:** Lớp trung gian giữa Domain và Data Mapping.

Chương 2

Phát triển Hệ thống E-Commerce dựa trên Microservices

2.1 Xác định yêu cầu hệ thống

Trong bối cảnh thị trường thương mại điện tử cạnh tranh khốc liệt, việc xây dựng một hệ thống có khả năng thích ứng nhanh và quy mô lớn là điều bắt buộc. Hệ thống được thiết kế để phục vụ cả khách hàng (Customer) và nhân viên quản trị (Staff), tích hợp trí tuệ nhân tạo (AI) để tối ưu hóa trải nghiệm người dùng.

2.1.1 Yêu cầu chức năng (Functional Requirements)

Hệ thống bao gồm các nhóm chức năng chính sau:

- **Quản lý người dùng và xác thực:** Cho phép khách hàng đăng ký, đăng nhập và quản lý thông tin cá nhân. Nhân viên có các quyền hạn riêng biệt theo vai trò (Role-based Access Control).
- **Quản lý danh mục sản phẩm:** Hiển thị và quản lý thông tin chi tiết các dòng Laptop và Mobile, bao gồm cấu hình kỹ thuật, giá cả và chương trình khuyến mãi.
- **Hệ thống giỏ hàng:** Cho phép người dùng thêm, bớt sản phẩm từ các dịch vụ khác nhau (Laptop/Mobile) vào một giỏ hàng nhất nhất quán.
- **Tư vấn thông minh (AI Chatbot):** Cung cấp khả năng trả lời tự động các thắc mắc về sản phẩm dựa trên cơ sở tri thức (Knowledge Base) được trích xuất từ cơ sở dữ liệu đồ thị.

2.1.2 Yêu cầu phi chức năng (Non-functional Requirements)

Để đảm bảo chất lượng dịch vụ cấp doanh nghiệp, hệ thống cần đáp ứng:

- **Tính mở rộng (Scalability):** Khả năng mở rộng độc lập các dịch vụ có tải cao như Laptop Service hoặc AI Service mà không ảnh hưởng đến toàn bộ hệ thống.
- **Tính sẵn sàng cao (High Availability):** Hệ thống phải hoạt động liên tục, giảm thiểu điểm chết duy nhất (Single Point of Failure) thông qua cơ chế replication và load balancing.
- **Tính bảo mật (Security):** Dữ liệu người dùng phải được mã hóa, các API được bảo vệ qua Gateway và cơ chế xác thực tập trung.
- **Khả năng bảo trì (Maintainability):** Mã nguồn được tổ chức theo cấu trúc chuẩn của Django và Microservices, giúp dễ dàng nâng cấp và sửa lỗi từng phần.

2.2 Phân rã hệ thống theo thiết kế hướng tên miền (DDD)

Việc phân rã hệ thống dựa trên nghiệp vụ thay vì kỹ thuật giúp đảm bảo tính bao đóng và giảm thiểu sự phụ thuộc chéo.

2.2.1 Xác định các Ngữ cảnh bao đóng (Bounded Contexts)

Dựa trên phân tích nghiệp vụ, hệ thống được chia thành 5 Bounded Contexts chính:

- **Customer Context:** Tập trung vào thực thể khách hàng và quy trình tương tác của họ với hệ thống (Login, Register, Cart).
- **Laptop Context:** Quản lý chuyên sâu về các sản phẩm máy tính xách tay với các đặc tính kỹ thuật riêng biệt (CPU, GPU, Screen).
- **Mobile Context:** Quản lý các thiết bị di động với các thuộc tính như Camera, dung lượng Pin.
- **Staff Context:** Quản lý nhân sự và các hoạt động quản trị nội bộ.
- **AI Context:** Cung cấp các khả năng thông minh, xử lý ngôn ngữ tự nhiên để tư vấn bán hàng.

2.2.2 Các nguyên tắc và ràng buộc kiến trúc

Hệ thống tuân thủ các ràng buộc nghiêm ngặt:

- **Database per Service:** Mỗi ngữ cảnh sở hữu một cơ sở dữ liệu riêng, đảm bảo tính cô lập hoàn toàn về dữ liệu.
- **Polyglot Persistence:** Sử dụng đa dạng các loại cơ sở dữ liệu (PostgreSQL cho dữ liệu quan hệ và Neo4j cho dữ liệu đồ thị phục vụ AI).
- **Giao tiếp qua API/Gateway:** Các dịch vụ không gọi trực tiếp cơ sở dữ liệu của nhau mà phải thông qua các giao diện lập trình ứng dụng (API).

2.3 Thiết kế và Triển khai Customer Service

Customer Service đóng vai trò là điểm chạm đầu tiên của người dùng, xử lý các nghiệp vụ liên quan đến định danh và quản lý trạng thái mua hàng.

2.3.1 Logic nghiệp vụ

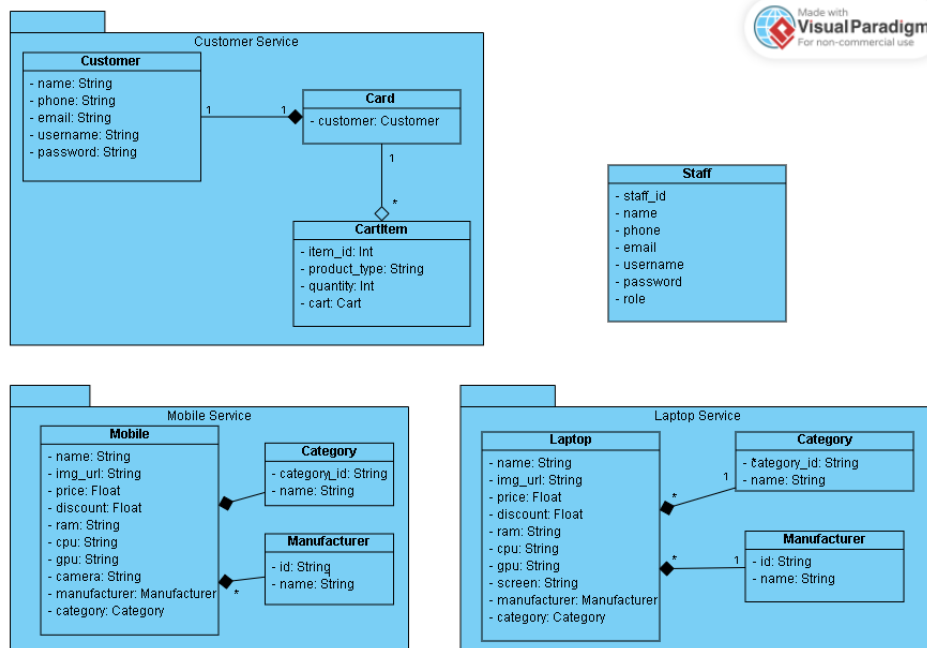
- **Quản lý danh tính:** Xử lý quy trình đăng ký tài khoản mới và xác thực người dùng dựa trên thông tin định danh duy nhất.
- **Quản lý giỏ hàng tập trung:** Khởi tạo giỏ hàng riêng biệt cho mỗi khách hàng ngay khi đăng ký.
- **Xử lý vật phẩm đa dạng:** Cho phép thêm các loại sản phẩm khác nhau (Laptop, Mobile) vào chung một giỏ hàng bằng cách lưu trữ mã định danh và loại sản phẩm.

- **Cập nhật số lượng thông minh:** Tự động kiểm tra sự tồn tại của vật phẩm trong giỏ; nếu đã tồn tại, hệ thống sẽ cộng dồn số lượng thay vì tạo bản ghi mới.

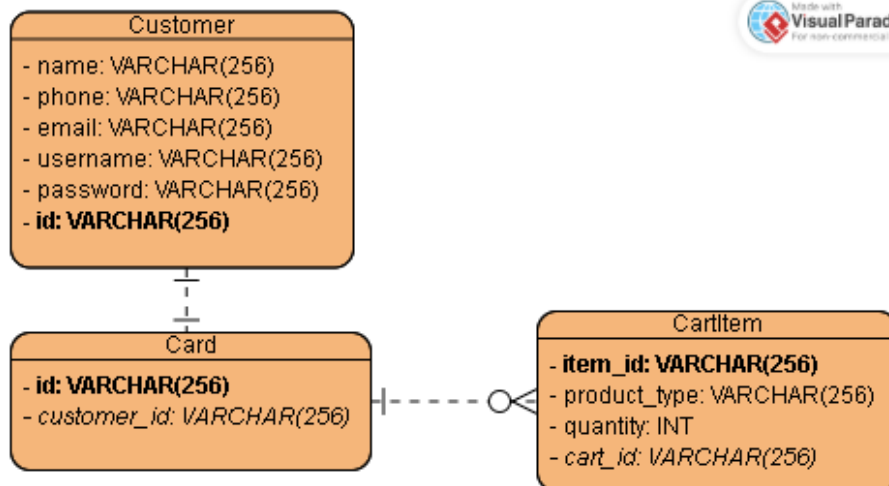
2.3.2 Mô hình dữ liệu (Data Models)

Hệ thống sử dụng các thực thể chính sau:

- **Customer:**
 - name: Họ và tên khách hàng (String).
 - phone: Số điện thoại liên lạc (String).
 - email: Địa chỉ thư điện tử (Unique Email).
 - username: Tên đăng nhập (Unique String).
 - password: Mật khẩu đã được băm (Hash String).
- **Cart:**
 - customer: Liên kết 1-1 với thực thể Customer.
- **CartItem:**
 - item_id: ID của sản phẩm từ dịch vụ ngoại lai (Integer).
 - product_type: Loại sản phẩm (MOBILE hoặc LAPTOP).
 - quantity: Số lượng sản phẩm trong giỏ (Positive Integer).
 - cart: Liên kết N-1 với thực thể Cart.



Hình 2.1: Sơ đồ lớp (Class Diagram)



Hình 2.2: Thiết kế Cơ sở dữ liệu Customer Service

2.3.3 Thiết kế CSDL

2.3.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: MySQL.

Lý do lựa chọn: MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) phổ biến, mã nguồn mở, có hiệu suất cao cho các tác vụ đọc/ghi thông tin người dùng và giỏ hàng. Nó đảm bảo tính nhất quán dữ liệu (ACID) cần thiết cho các giao dịch liên quan đến khách hàng.

Mã nguồn định nghĩa các bảng (Django Models):

```

class Customer(models.Model):
    name = models.CharField(max_length=255)
    phone = models.CharField(max_length=20)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=150, unique=True)
    password = models.CharField(max_length=255)

class Cart(models.Model):
    customer = models.OneToOneField(Customer, on_delete=models.CASCADE)

class CartItem(models.Model):
    item_id = models.IntegerField()
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)
    product_type = models.CharField(max_length=10,
                                     choices=(('MOBILE', 'MOBILE'), ('LAPTOP', 'LAPTOP'))
  
```

Phương pháp Seed dữ liệu: Hệ thống sử dụng Django Management Command để khởi tạo dữ liệu mẫu. Lệnh thực hiện:

```
python manage.py seed_db
```

Dữ liệu được tạo bao gồm các tài khoản khách hàng mặc định và giỏ hàng trống tương ứng để sẵn sàng cho việc mua sắm.

2.3.5 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/register/	POST	Đăng ký tài khoản	Thông tin Customer	JSON khách hàng
/login/	POST	Xác thực người dùng	Username, Password	Thông tin khách hàng
/cart-items/	GET	Liệt kê vật phẩm	Token xác thực	List CartItems
/cart-items/	POST	Thêm vào giỏ	item_id, type, qty	CartItem chi tiết

2.4 Thiết kế và Triển khai Laptop Service

Dịch vụ quản lý kho dữ liệu và đặc tính kỹ thuật của các dòng máy tính xách tay.

2.4.1 Logic nghiệp vụ

- **Phân loại sản phẩm:** Tổ chức máy tính theo hãng sản xuất và danh mục sử dụng (Gaming, Văn phòng, v.v.).
- **Quản lý cấu hình chi tiết:** Lưu trữ các thông số phần cứng chuyên sâu phục vụ việc so sánh và tra cứu.
- **Đồng bộ hóa dữ liệu đồ thị:** Cung cấp dữ liệu để ánh xạ sang Neo4j, xây dựng mạng lưới liên kết giữa sản phẩm và các thành phần phần cứng.

2.4.2 Mô hình dữ liệu (Data Models)

- **Mô hình quan hệ (Relational Model):**
 - **Laptop:** name, img_url, price, discount, ram, cpu, gpu, screen.
 - **Manufacturer / Category:** Chỉ bao gồm trường name để định danh.
- **Mô hình đồ thị (Graph Model):**
 - **Nodes:** Laptop, CPU, GPU, RAM, Screen, Manufacturer.
 - **Relationships:** HAS_CPU, MADE_BY, BELONGS_TO.

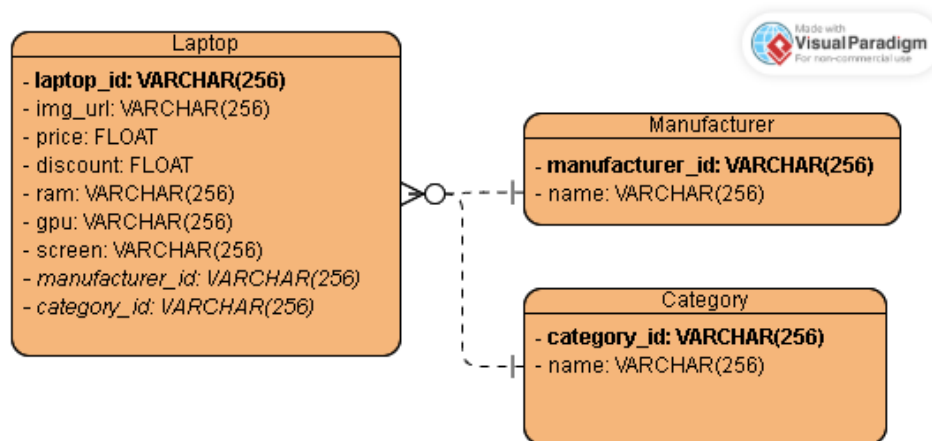
2.4.3 Thiết kế CSDL

2.4.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: PostgreSQL (Relational) và Neo4j (Graph).

Lý do lựa chọn:

- **PostgreSQL:** Được sử dụng để lưu trữ dữ liệu sản phẩm có cấu trúc chặt chẽ. PostgreSQL hỗ trợ tốt các kiểu dữ liệu phức tạp và khả năng mở rộng mạnh mẽ cho danh mục sản phẩm lớn.



Hình 2.3: Thiết kế Cơ sở dữ liệu Laptop Service

- **Neo4j:** Được sử dụng để lưu trữ các mối quan hệ kỹ thuật giữa Laptop và phần cứng (CPU, GPU, RAM). Điều này cho phép AI Service truy vấn và so sánh cấu hình một cách nhanh chóng thông qua các thuật toán đồ thị.

Mã nguồn định nghĩa các bảng (Django Models):

```

class Manufacturer(models.Model):
    name = models.CharField(max_length=255)

class Category(models.Model):
    name = models.CharField(max_length=255)

class Laptop(models.Model):
    name = models.CharField(max_length=255)
    img_url = models.URLField(max_length=500)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    manufacturer = models.ForeignKey(Manufacturer, on_delete=models.CASCADE)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    ram = models.CharField(max_length=50)
    cpu = models.CharField(max_length=255)
    gpu = models.CharField(max_length=255)
    screen = models.CharField(max_length=255)
  
```

Phương pháp Seed dữ liệu: Dữ liệu được khởi tạo thông qua script Python độc lập. Lệnh thực hiện:

```
python seed_db.py
```

Script này sẽ tự động tạo các bản ghi cho Hãng sản xuất, Danh mục và danh sách các Laptop mẫu cùng thông số kỹ thuật chi tiết.

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/laptops/	GET	Lấy danh sách sản phẩm	Tham số lọc (tùy chọn)	Danh sách Laptop
/laptops/{id}/	GET	Chi tiết sản phẩm	ID sản phẩm	Đối tượng Laptop
/manufacturers/	GET	Lấy danh sách hãng	Không	Danh sách hãng

2.4.5 Giao diện lập trình ứng dụng (API Endpoints)

2.5 Thiết kế và Triển khai Mobile Service

Quản lý dải sản phẩm thiết bị di động và các đặc thù công nghệ liên quan.

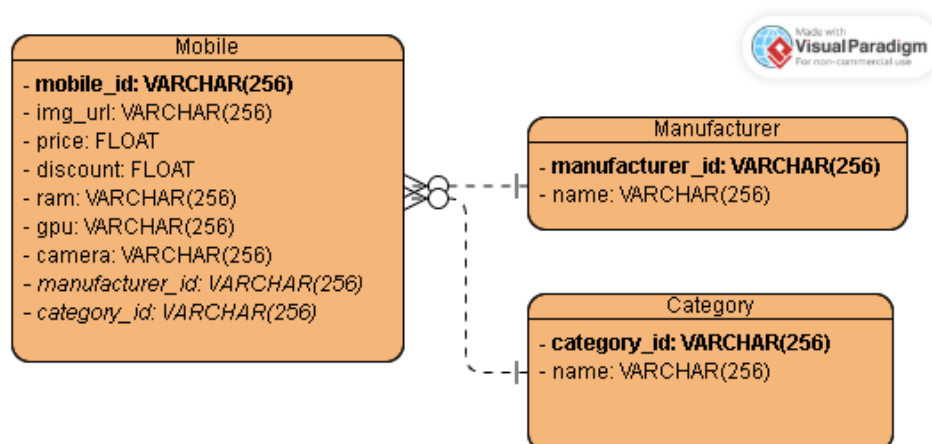
2.5.1 Logic nghiệp vụ

- **Quản lý thông số di động:** Tập trung vào các yếu tố người dùng quan tâm như Camera và hiệu năng xử lý.
- **Phân mảnh thị trường:** Hỗ trợ phân loại sản phẩm theo thương hiệu để người dùng dễ dàng tiếp cận.

2.5.2 Mô hình dữ liệu (Data Models)

- **Mobile:**
 - name, price, discount, camera: Thuộc tính cơ bản và đặc thù.
 - ram, cpu, gpu: Cấu hình hiệu năng.
 - manufacturer, category: Tham chiếu thực thể cha.

2.5.3 Thiết kế CSDL



Hình 2.4: Thiết kế Cơ sở dữ liệu Mobile Service

2.5.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: PostgreSQL và Neo4j.

Lý do lựa chọn: Tương tự như Laptop Service, Mobile Service sử dụng PostgreSQL cho các thuộc tính thương mại và Neo4j cho các mối quan hệ kỹ thuật. Sự kết hợp này giúp hệ thống vừa đảm bảo tính nhất quán giao dịch, vừa tối ưu cho các tính năng thông minh của AI Chatbot khi cần so sánh các thông số như Camera hay dung lượng Pin giữa các dòng điện thoại.

Mã nguồn định nghĩa các bảng (Django Models):

```
class Manufacturer(models.Model):
    name = models.CharField(max_length=255)

class Category(models.Model):
    name = models.CharField(max_length=255)

class Mobile(models.Model):
    name = models.CharField(max_length=255)
    img_url = models.URLField(max_length=500)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    manufacturer = models.ForeignKey(Manufacturer, on_delete=models.CASCADE)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    ram = models.CharField(max_length=50)
    cpu = models.CharField(max_length=255)
    gpu = models.CharField(max_length=255)
    camera = models.CharField(max_length=255)
```

Phương pháp Seed dữ liệu: Sử dụng Django Management Command để khởi tạo danh mục điện thoại. Lệnh thực hiện:

```
python manage.py seed_db
```

Dữ liệu mẫu bao gồm các dòng Flagship phổ biến như iPhone, Samsung Galaxy, Xiaomi,... cùng các thông số phần cứng tương ứng.

2.5.5 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/mobiles/	GET	Danh sách điện thoại	Không	List Mobile
/mobiles/{id}/	GET	Xem chi tiết	ID	Chi tiết Mobile

2.6 Thiết kế và Triển khai Staff Service

Hệ thống quản trị dành cho nhân sự vận hành sản phẩm thương mại điện tử.

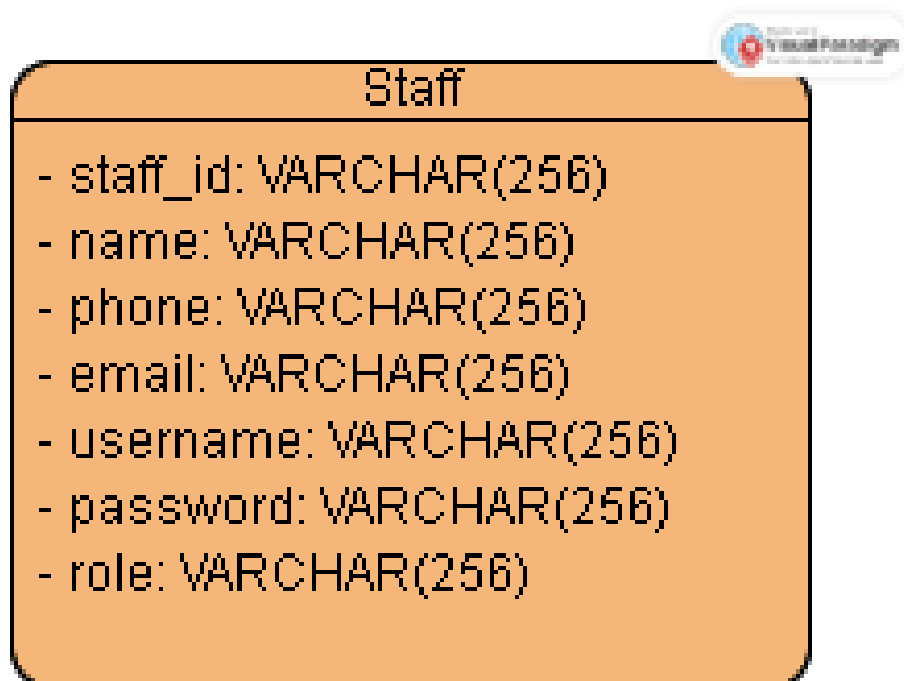
2.6.1 Logic nghiệp vụ

- **Kiểm soát truy cập (RBAC):** Phân quyền nhân viên theo các vai trò khác nhau trong hệ thống.
- **Quản lý vận hành:** Cung cấp giao diện để nhân viên theo dõi trạng thái hệ thống và dữ liệu sản phẩm.

2.6.2 Mô hình dữ liệu (Data Models)

- **Staff:**
 - staff_id: Mã nhân viên duy nhất.
 - name, phone, email: Thông tin liên lạc cá nhân.
 - username, password: Thông tin đăng nhập.
 - role: Vai trò (Admin, Manager, v.v.).

2.6.3 Thiết kế CSDL



Hình 2.5: Thiết kế Cơ sở dữ liệu Staff Service

2.6.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: MySQL.

Lý do lựa chọn: MySQL cung cấp cơ chế bảo mật và quản lý người dùng mạnh mẽ, phù hợp để lưu trữ thông tin nhân sự và quyền hạn truy cập hệ thống. Tính ổn định và khả năng xử lý đồng thời của MySQL giúp nhân viên thực hiện các thao tác quản trị một cách mượt mà.

Mã nguồn định nghĩa các bảng (Django Models):

```
class Staff(models.Model):
    staff_id = models.CharField(max_length=50, unique=True)
    name = models.CharField(max_length=255)
    phone = models.CharField(max_length=20)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=150, unique=True)
    password = models.CharField(max_length=255)
    role = models.CharField(max_length=100)
```

Phương pháp Seed dữ liệu: Lệnh thực hiện:

```
python manage.py seed_db
```

Dữ liệu khởi tạo bao gồm tài khoản Quản trị hệ thống (System Administrator) và các nhân viên mẫu với các vai trò khác nhau (Admin, Manager, Staff) để phục vụ việc kiểm thử phân quyền.

2.6.5 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/login/	POST	Đăng nhập quản trị	Credentials	Staff Profile

Chương 3

AI Service

3.1 Mục đích của AI Service

AI Service trong hệ thống được xây dựng nhằm mục đích tối ưu hóa quy trình tư vấn khách hàng. Thay vì nhân viên phải trực tiếp trả lời các thắc mắc lặp đi lặp lại về thông số kỹ thuật, AI Service đóng vai trò là một trợ lý ảo thông minh:

- **Tư vấn 24/7:** Phản hồi khách hàng ngay lập tức tại mọi thời điểm.
- **Độ chính xác cao:** Cung cấp thông tin dựa trên dữ liệu thực tế được trích xuất từ kho hàng.
- **Cá nhân hóa trải nghiệm:** Hiểu ý định người dùng để đưa ra các gợi ý sản phẩm phù hợp.

3.2 Kiến trúc tổng quan AI Service

Dịch vụ được xây dựng trên nền tảng Django REST Framework, tích hợp trực tiếp các thư viện học máy hiện đại:

- **Hugging Face Transformers:** Thư viện lõi để triển khai và chạy mô hình ngôn ngữ lớn (LLM).
- **Neomodel:** Thư viện Object-Graph Mapping (OGM) để giao tiếp với cơ sở dữ liệu đồ thị Neo4j.
- **PyTorch:** Backend tính toán cho mô hình AI.
- **Background Loading:** Mô hình AI được tải trong một tiến trình chạy ngầm (threading) để không làm gián đoạn quá trình khởi động của server.

3.3 Logic hoạt động của Chatbot

Quy trình xử lý một yêu cầu của chatbot diễn ra theo các bước sau:

1. **Tiếp nhận:** API nhận `user_message` từ client qua phương thức POST.
2. **Trích xuất tri thức:** Hệ thống gọi hàm `get_knowledge_base()` để lấy toàn bộ thông tin sản phẩm (Laptop và Mobile) từ Neo4j.
3. **Xây dựng ngữ cảnh (Context):** Dữ liệu thô từ database được định dạng thành văn bản liệt kê tên sản phẩm, giá và cấu hình.

4. **Tạo Prompt:** Kết hợp ngữ cảnh sản phẩm và câu hỏi của người dùng vào một cấu trúc template quy định vai trò của trợ lý.
5. **Suy luận (Inference):** Mô hình AI xử lý prompt và sinh câu trả lời.
6. **Phản hồi:** Câu trả lời được tách khỏi định dạng prompt và gửi về cho người dùng.

3.4 Kiến trúc Mô hình AI sử dụng

Mô hình: TinyLlama/TinyLlama-1.1B-Chat-v1.0.

Cấu hình kỹ thuật:

- **Device:** Chạy trên CPU thay vì GPU.
- **Dtype:** `torch.float32` để đảm bảo tính tương thích và chính xác trên CPU.
- **Max New Tokens:** 64 tokens (giới hạn độ dài câu trả lời để tăng tốc độ phản hồi).
- **Decoding Strategy:** `do_sample=False` (sử dụng Greedy Search để câu trả lời mang tính nhất quán và ổn định nhất).

Lý do lựa chọn và Đánh đổi: TinyLlama là mô hình nhỏ gọn với 1,1 tỷ tham số. Việc sử dụng mô hình này giúp hệ thống có thể chạy mượt mà trên môi trường containerized với tài nguyên CPU hạn chế mà không cần đến phần cứng chuyên dụng (GPU). Đánh đổi lại là khả năng lập luận phức tạp của mô hình sẽ hạn chế hơn so với các mô hình lớn (như Llama-2 70B), nhưng hoàn toàn đáp ứng tốt nhu cầu tư vấn thông số sản phẩm đơn giản.

3.5 Knowledge Base và Neo4j

Cấu trúc tri thức: AI Service sử dụng mô hình dữ liệu đồ thị để biểu diễn sản phẩm.

- **Nodes:** Bao gồm các thực thể như Laptop, Mobile, CPU, GPU, RAM, Manufacturer.
- **Relationships:** Các mối quan hệ như HAS_CPU, MADE_BY, HAS_CAMERA tạo thành một mạng lưới tri thức liên kết.

Tại sao sử dụng Neo4j? Khác với SQL, Neo4j cho phép AI Service "nhìn" thấy các mối quan hệ giữa các linh kiện một cách trực tiếp. Điều này cực kỳ hữu ích khi chatbot cần thực hiện các truy vấn so sánh hoặc tìm kiếm sản phẩm có chung một đặc tính kỹ thuật cụ thể (ví dụ: "Tìm các laptop dùng chung CPU Intel Core i7").

3.6 Kỹ thuật RAG (Retrieval-Augmented Generation)

AI Service áp dụng kỹ thuật RAG đơn giản để khắc phục hiện tượng "ảo giác" (hallucination) của mô hình AI. Thay vì để mô hình tự nhớ thông số sản phẩm (vốn dễ sai lệch), hệ thống thực hiện:

- **Retrieval:** Truy xuất thông tin thực tế từ Neo4j ngay tại thời điểm người dùng đặt câu hỏi.

- **Augmentation:** Bơm dữ liệu vừa truy xuất được vào `system_prompt` dưới dạng "Tri thức nền tảng".
- **Generation:** Yêu cầu mô hình AI chỉ sinh câu trả lời dựa trên những thông tin đã cung cấp.

Cơ chế này đảm bảo chatbot không bao giờ tư vấn những sản phẩm không có trong kho hàng hoặc sai lệch về giá cả.

3.7 Kiến trúc API Gateway và Điều phối

Để che giấu sự phức tạp của hệ thống microservices phía sau, API Gateway được triển khai như một facade duy nhất. Nó chịu trách nhiệm proxy các yêu cầu đến đúng dịch vụ mục tiêu dựa trên đường dẫn URL, đồng thời xử lý các vấn đề xuyên suốt (cross-cutting concerns) như timeout và headers mapping.